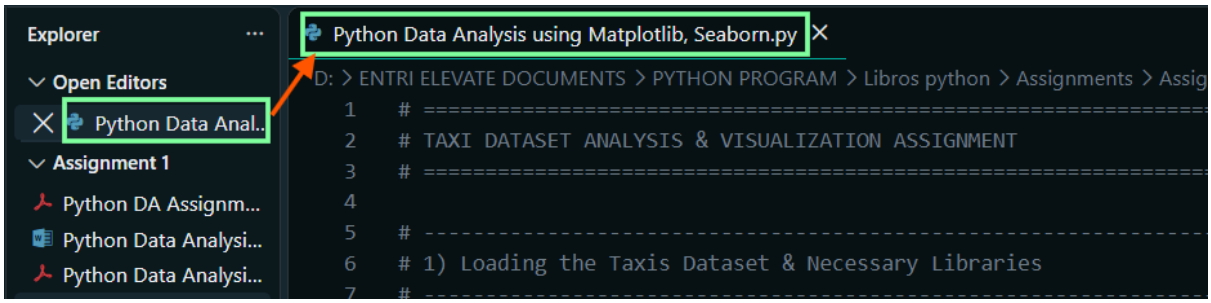
PYTHON DA ASSIGNMENT 2 – PYTHON

MATPLOTLIB, SEABORN

TOOLS: Python / VS Code
PROJECT TITLE: Python DA - Matplotlib, Seaborn
PROJECT OBJECTIVE: To clean, analyze, and visualize taxi trip data using Python, Matplotlib, and Seaborn to identify fare trends, trip distance patterns, pickup borough activity, and customer payment behavior.

DATE OF SUBMISSION: 07/11/2026 – 4:30 PM

PROGRAMMING FILE NAME - "Python Data Analysis using Matplotlib, Seaborn.py"



1) LOADING THE TAXIS DATASET AND NECESSARY LIBRARIES

```
# 1) Loading the Taxis Dataset & Necessary Libraries
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
```

1.1) SET AESTHETICS FOR CLEAN VISUALIZATION OUTPUT

```
# 1.1) Set aesthetics for clean visualization output
sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (10, 6)
plt.rcParams["font.size"] = 10
```

1.2) LOAD THE SEABORN 'TAXIS' DATASET

```
# 1.2) Load the Seaborn 'taxis' dataset
df = sns.load_dataset("taxis")
```

1.3) Display initial dataset information

```
#1.3) Display initial dataset information
print("--- Initial Dataset Info ---")
print(f"Dataset Shape: {df.shape}")
print("\nFirst 5 Rows:")
print(df.head())
```

2) HANDLING MISSING VALUES

2.1) DISPLAY MISSING VALUES PER COLUMN

```
# 2.1) Display missing values per column
print("\n--- Missing Value Analysis ---")
```

2.2) COUNT MISSING VALUES PER COLUMN

```
# 2.2) Count missing values per column
missing_before = df.isnull().sum()
print("Missing values per column prior to cleaning:")
print(missing_before[missing_before > 0])
```

2.3) STRATEGY IMPLEMENTATION

2.3.1) IMPUTE NUMERICAL MISSING VALUES USING MEDIAN

```
# 2.3.1) Impute numerical missing values using Median (robust to taxi
fare/distance outliers)
df["distance"] = df["distance"].fillna(df["distance"].median())
df["fare"] = df["fare"].fillna(df["fare"].median())
df["tip"] = df["tip"].fillna(df["tip"].median())
df["tolls"] = df["tolls"].fillna(df["tolls"].median())
df["total"] = df["total"].fillna(df["total"].median())
```

2.3.2) IMPUTE CATEGORICAL MISSING VALUES USING MODE

```
# 2.3.2) Impute categorical missing values using Mode (most frequent value)
df["payment"] = df["payment"].fillna(df["payment"].mode()[0])
```

2.3.3) DROP ROWS WITH MISSING BOROUGH / ZONE DATA

```
# 2.3.3) Critical borough/zone columns: Drop rows where location metadata is
completely missing
# Imputing borough/zones could introduce false geospatial patterns.
df = df.dropna(subset=["pickup_borough", "dropoff_borough", "pickup_zone",
"dropoff_zone"]).copy()
```

2.4) FINAL CHECK FOR REMAINING MISSING VALUES

```
# 2.4) Final check for any remaining missing values
print("\nMissing values remaining after cleaning:")
print(df.isnull().sum().sum())
print(f"Cleaned Dataset Shape: {df.shape}")
```

3) VISUALIZATIONS USING MATPLOTLIB / PANDAS PLOT

3.1) LINE CHART: FARE OVER TIME

```
# 3.1) Line Chart: Fare over time
# Ensure pickup column is converted to datetime format
df["pickup"] = pd.to_datetime(df["pickup"])
```

3.1.1) GROUP BY DATE FOR DAILY FARE PROGRESSION

```
# 3.1.1)Group by date to show smooth daily fare progression over time
daily_fare = df.groupby(df["pickup"].dt.date)["fare"].mean()
```

3.1.2) PLOT THE DAILY AVERAGE FARE OVER TIME

```
# 3.1.2) Plotting the daily average fare over time
plt.figure(figsize=(12, 5))
plt.plot(daily_fare.index, daily_fare.values, marker="o", color="#1f77b4",
linewidth=2)
plt.title("Average Taxi Fare Over Time", fontsize=14, fontweight="bold")
plt.xlabel("Pickup Date", fontsize=12)
plt.ylabel("Mean Fare ($)", fontsize=12)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

3.2) BAR CHART: TOTAL FARE FOR EACH PICKUP BOROUGH

```
# 3.2) Bar Chart: Total fare for each pickup_borough
total_fare_borough =
df.groupby("pickup_borough")["fare"].sum().sort_values(ascending=False)
```

3.2.1) PLOT TOTAL FARE BY PICKUP BOROUGH

```
# 3.2.1) Plotting total fare by pickup borough
plt.figure(figsize=(9, 5))
total_fare_borough.plot(kind="bar", color="#2ca02c", edgecolor="black")
plt.title("Total Revenue Generated by Pickup Borough", fontsize=14,
fontweight="bold")
plt.xlabel("Pickup Borough", fontsize=12)
plt.ylabel("Total Fare ($)", fontsize=12)
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

3.3) PIE CHART: TRIPS BY PAYMENT METHOD

```
# 3.3) Pie Chart: Distribution of trips based on payment method
payment_counts = df["payment"].value_counts()
```

3.3.1) PLOT THE PAYMENT METHOD PIE CHART

```
# 3.3.1) Plotting the pie chart for payment method distribution
plt.figure(figsize=(7, 7))
plt.pie(
```

```
    payment_counts,
    labels=payment_counts.index.str.title(),
    autopct="%1.1f%%",
    startangle=140,
    colors=["#4c72b0", "#dd8452"],
    explode=(0.05, 0),
    shadow=True,
)
plt.title("Trip Distribution by Payment Method", fontsize=14,
fontweight="bold")
plt.tight_layout()
plt.show()
```

3.4) HISTOGRAM: DISTRIBUTION OF TRIP DISTANCE

```
# 3.4) Histogram: Distribution of Trip Distance
plt.figure(figsize=(10, 5))
plt.hist(df["distance"], bins=30, color="#8172b3", edgecolor="black",
alpha=0.8)
plt.title("Distribution of Trip Distances", fontsize=14, fontweight="bold")
plt.xlabel("Distance (Miles)", fontsize=12)
plt.ylabel("Frequency (Trip Count)", fontsize=12)
plt.xlim(0, df["distance"].max())
plt.tight_layout()
plt.show()
```

3.5) BOX PLOT: TIP DISTRIBUTION BY PICKUP BOROUGH

```
# 3.5) Box Plot: Tip distribution for each pickup_borough
plt.figure(figsize=(10, 6))
df.boxplot(column="tip", by="pickup_borough", grid=True, patch_artist=True)
plt.title("Tip Distribution Across Pickup Boroughs", fontsize=14,
fontweight="bold")
plt.suptitle("")  # Remove default pandas boxplot title
plt.xlabel("Pickup Borough", fontsize=12)
plt.ylabel("Tip Amount ($)", fontsize=12)
plt.tight_layout()
plt.show()
```

4) VISUALIZATIONS USING SEABORN

4.1) COUNT PLOT: NUMBER OF TRIPS IN EACH PICKUP BOROUGH

```
# 4.1) Count Plot: Number of trips in each pickup_borough
plt.figure(figsize=(9, 5))
sns.countplot(
    data=df,
```

```
    x="pickup_borough",
    order=df["pickup_borough"].value_counts().index,
    palette="viridis",
)
plt.title("Total Trip Volume by Pickup Borough", fontsize=14,
fontweight="bold")
plt.xlabel("Pickup Borough", fontsize=12)
plt.ylabel("Number of Trips", fontsize=12)
plt.tight_layout()
plt.show()
```

4.2) SCATTER PLOT: DISTANCE VS FARE BY PICKUP BOROUGH

```
# 4.2) Scatter Plot: Relationship between Distance and Fare by Pickup Borough
plt.figure(figsize=(10, 6))
sns.scatterplot(
    data=df,
    x="distance",
    y="fare",
    hue="pickup_borough",
    alpha=0.7,
    s=60,
    palette="Set2",
)
plt.title("Trip Distance vs. Fare Amount by Borough", fontsize=14,
fontweight="bold")
plt.xlabel("Distance (Miles)", fontsize=12)
plt.ylabel("Fare ($)", fontsize=12)
plt.legend(title="Pickup Borough")
plt.tight_layout()
plt.show()
```

4.3) HEATMAP: CORRELATION BETWEEN NUMERICAL VARIABLES

```
# 4.3) Heatmap: Correlation between numerical variables
plt.figure(figsize=(8, 6))
num_cols = ["distance", "fare", "tip", "tolls", "total"]
corr_matrix = df[num_cols].corr()
sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap="coolwarm", vmin=-1,
vmax=1, linewidths=0.5)
plt.title("Correlation Matrix of Numeric Trip Features", fontsize=14,
fontweight="bold")
plt.tight_layout()
plt.show()
```

4.4) PAIR PLOT: PAIRWISE RELATIONSHIPS BY PICKUP ZONE

```
# 4.4) Pair Plot: Pairwise relationships colored by pickup_zone (Top 5 zones for visual clarity)
top_zones = df["pickup_zone"].value_counts().nlargest(5).index
df_top_zones = df[df["pickup_zone"].isin(top_zones)]

sns.pairplot(
    df_top_zones,
    vars=["distance", "fare", "tip", "total"],
    hue="pickup_zone",
    palette="tab10",
    diag_kind="kde",
    plot_kws={"alpha": 0.6, "s": 30},
)
plt.suptitle("Pairwise Relationships for Top 5 Pickup Zones", y=1.02, fontsize=14, fontweight="bold")
plt.show()
```

4.5) VIOLIN PLOT: FARE BY PAYMENT METHOD

```
# 4.5) Violin Plot: Distribution of Fare by Payment Method
plt.figure(figsize=(9, 5))
sns.violinplot(
    data=df,
    x="payment",
    y="fare",
    palette="Pastel1",
    inner="quartile",
    cut=0,
)
plt.title("Fare Amount Distribution Across Payment Methods", fontsize=14, fontweight="bold")
plt.xlabel("Payment Method", fontsize=12)
plt.ylabel("Fare ($)", fontsize=12)
plt.tight_layout()
plt.show()
```

COMPLETE PROGRAM

```
#
# ============================================================================
# TAXI DATASET ANALYSIS & VISUALIZATION ASSIGNMENT
#
# ============================================================================

# 1) Loading the Taxis Dataset & Necessary Libraries

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

# 1.1) Set aesthetics for clean visualization output
sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (10, 6)
plt.rcParams["font.size"] = 10

# 1.2) Load the Seaborn 'taxis' dataset
df = sns.load_dataset("taxis")

#1.3) Display initial dataset information
print("--- Initial Dataset Info ---")
print(f"Dataset Shape: {df.shape}")
print("\nFirst 5 Rows:")
print(df.head())


# 2) Handling Missing Values

# 2.1) Display missing values per column
print("\n--- Missing Value Analysis ---")
# 2.2) Count missing values per column
missing_before = df.isnull().sum()
print("Missing values per column prior to cleaning:")
print(missing_before[missing_before > 0])

# 2.3) Strategy Implementation:
# 2.3.1) Impute numerical missing values using Median (robust to taxi
fare/distance outliers)
df["distance"] = df["distance"].fillna(df["distance"].median())
df["fare"] = df["fare"].fillna(df["fare"].median())
df["tip"] = df["tip"].fillna(df["tip"].median())
df["tolls"] = df["tolls"].fillna(df["tolls"].median())
df["total"] = df["total"].fillna(df["total"].median())

# 2.3.2) Impute categorical missing values using Mode (most frequent value)
```

```
df["payment"] = df["payment"].fillna(df["payment"].mode()[0])

# 2.3.3) Critical borough/zone columns: Drop rows where location metadata is
completely missing
# Imputing borough/zones could introduce false geospatial patterns.
df = df.dropna(subset=["pickup_borough", "dropoff_borough", "pickup_zone",
"dropoff_zone"]).copy()

# 2.4) Final check for any remaining missing values
print("\nMissing values remaining after cleaning:")
print(df.isnull().sum().sum())
print(f"Cleaned Dataset Shape: {df.shape}")


# 3) Visualizations using Matplotlib / Pandas Plot

# 3.1) Line Chart: Fare over time
# Ensure pickup column is converted to datetime format
df["pickup"] = pd.to_datetime(df["pickup"])

# 3.1.1)Group by date to show smooth daily fare progression over time
daily_fare = df.groupby(df["pickup"].dt.date)["fare"].mean()

# 3.1.2) Plotting the daily average fare over time
plt.figure(figsize=(12, 5))
plt.plot(daily_fare.index, daily_fare.values, marker="o", color="#1f77b4",
linewidth=2)
plt.title("Average Taxi Fare Over Time", fontsize=14, fontweight="bold")
plt.xlabel("Pickup Date", fontsize=12)
plt.ylabel("Mean Fare ($)", fontsize=12)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()


# 3.2) Bar Chart: Total fare for each pickup_borough
total_fare_borough =
df.groupby("pickup_borough")["fare"].sum().sort_values(ascending=False)

# 3.2.1) Plotting total fare by pickup borough
plt.figure(figsize=(9, 5))
total_fare_borough.plot(kind="bar", color="#2ca02c", edgecolor="black")
plt.title("Total Revenue Generated by Pickup Borough", fontsize=14,
fontweight="bold")
plt.xlabel("Pickup Borough", fontsize=12)
plt.ylabel("Total Fare ($)", fontsize=12)
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

```python
# 3.3) Pie Chart: Distribution of trips based on payment method
payment_counts = df["payment"].value_counts()

# 3.3.1) Plotting the pie chart for payment method distribution
plt.figure(figsize=(7, 7))
plt.pie(
    payment_counts,
    labels=payment_counts.index.str.title(),
    autopct="%1.1f%%",
    startangle=140,
    colors=["#4c72b0", "#dd8452"],
    explode=(0.05, 0),
    shadow=True,
)
plt.title("Trip Distribution by Payment Method", fontsize=14,
fontweight="bold")
plt.tight_layout()
plt.show()


# 3.4) Histogram: Distribution of Trip Distance
plt.figure(figsize=(10, 5))
plt.hist(df["distance"], bins=30, color="#8172b3", edgecolor="black",
alpha=0.8)
plt.title("Distribution of Trip Distances", fontsize=14, fontweight="bold")
plt.xlabel("Distance (Miles)", fontsize=12)
plt.ylabel("Frequency (Trip Count)", fontsize=12)
plt.xlim(0, df["distance"].max())
plt.tight_layout()
plt.show()


# 3.5) Box Plot: Tip distribution for each pickup_borough
plt.figure(figsize=(10, 6))
df.boxplot(column="tip", by="pickup_borough", grid=True, patch_artist=True)
plt.title("Tip Distribution Across Pickup Boroughs", fontsize=14,
fontweight="bold")
plt.suptitle("")  # Remove default pandas boxplot title
plt.xlabel("Pickup Borough", fontsize=12)
plt.ylabel("Tip Amount ($)", fontsize=12)
plt.tight_layout()
plt.show()


# 4) Visualizations using Seaborn

# 4.1) Count Plot: Number of trips in each pickup_borough
```

```
plt.figure(figsize=(9, 5))
sns.countplot(
    data=df,
    x="pickup_borough",
    order=df["pickup_borough"].value_counts().index,
    palette="viridis",
)
plt.title("Total Trip Volume by Pickup Borough", fontsize=14,
fontweight="bold")
plt.xlabel("Pickup Borough", fontsize=12)
plt.ylabel("Number of Trips", fontsize=12)
plt.tight_layout()
plt.show()


# 4.2) Scatter Plot: Relationship between Distance and Fare by Pickup Borough
plt.figure(figsize=(10, 6))
sns.scatterplot(
    data=df,
    x="distance",
    y="fare",
    hue="pickup_borough",
    alpha=0.7,
    s=60,
    palette="Set2",
)
plt.title("Trip Distance vs. Fare Amount by Borough", fontsize=14,
fontweight="bold")
plt.xlabel("Distance (Miles)", fontsize=12)
plt.ylabel("Fare ($)", fontsize=12)
plt.legend(title="Pickup Borough")
plt.tight_layout()
plt.show()


# 4.3) Heatmap: Correlation between numerical variables
plt.figure(figsize=(8, 6))
num_cols = ["distance", "fare", "tip", "tolls", "total"]
corr_matrix = df[num_cols].corr()
sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap="coolwarm", vmin=-1,
vmax=1, linewidths=0.5)
plt.title("Correlation Matrix of Numeric Trip Features", fontsize=14,
fontweight="bold")
plt.tight_layout()
plt.show()

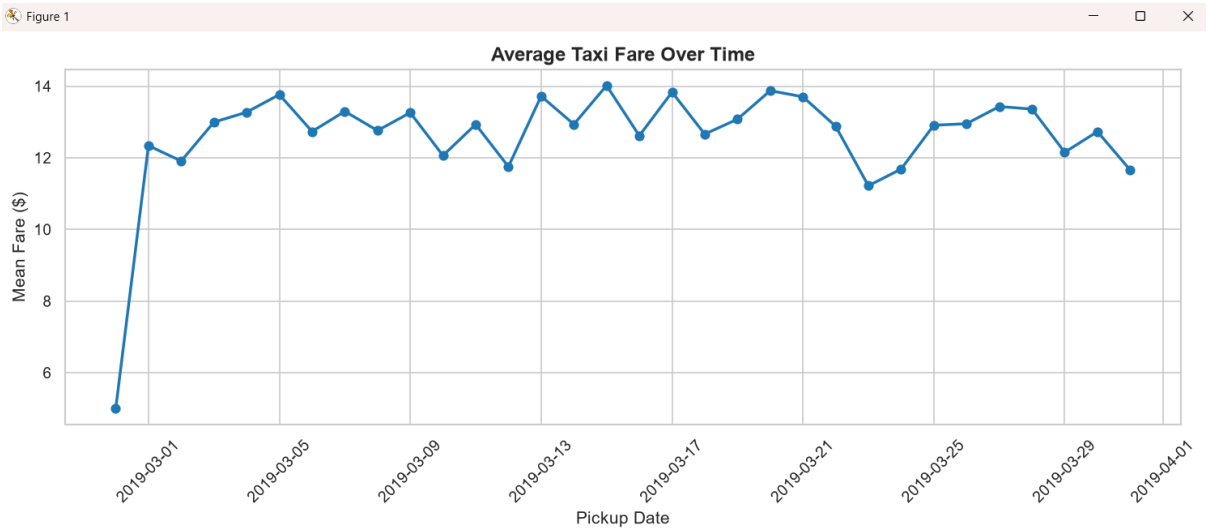

# 4.4) Pair Plot: Pairwise relationships colored by pickup_zone (Top 5 zones
for visual clarity)
```

```
top_zones = df["pickup_zone"].value_counts().nlargest(5).index
df_top_zones = df[df["pickup_zone"].isin(top_zones)]

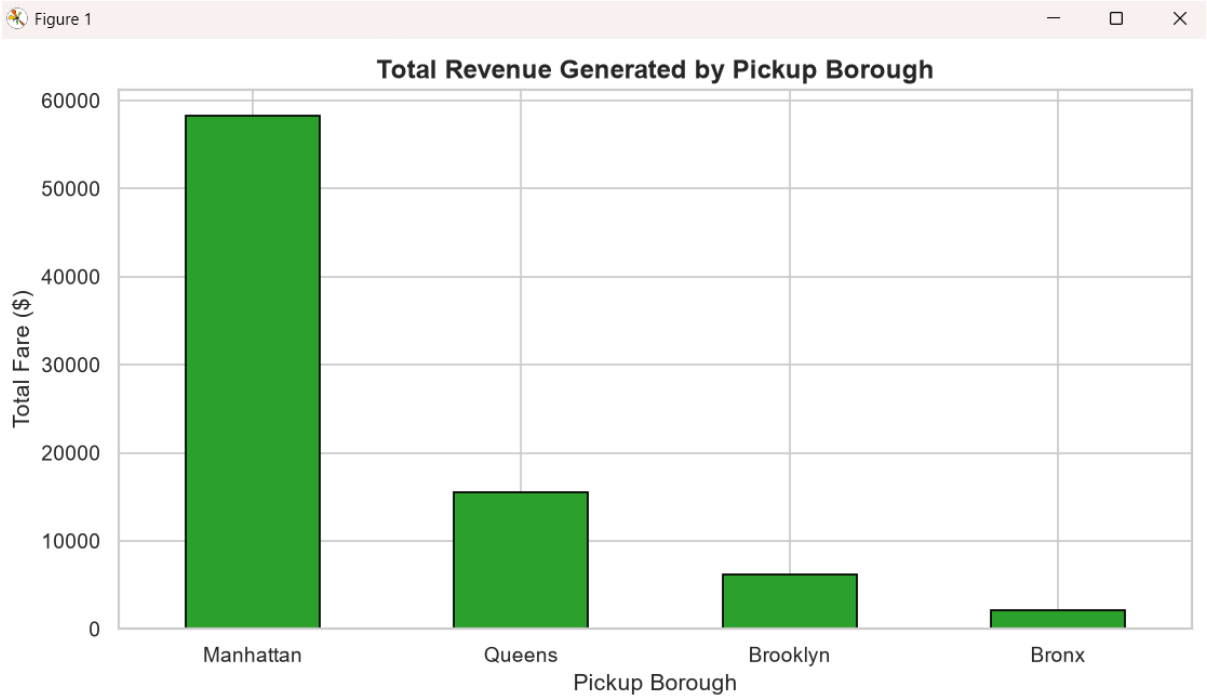
sns.pairplot(
    df_top_zones,
    vars=["distance", "fare", "tip", "total"],
    hue="pickup_zone",
    palette="tab10",
    diag_kind="kde",
    plot_kws={"alpha": 0.6, "s": 30},
)
plt.suptitle("Pairwise Relationships for Top 5 Pickup Zones", y=1.02,
fontsize=14, fontweight="bold")
plt.show()


# 4.5) Violin Plot: Distribution of Fare by Payment Method
plt.figure(figsize=(9, 5))
sns.violinplot(
    data=df,
    x="payment",
    y="fare",
    palette="Pastel1",
    inner="quartile",
    cut=0,
)
plt.title("Fare Amount Distribution Across Payment Methods", fontsize=14,
fontweight="bold")
plt.xlabel("Payment Method", fontsize=12)
plt.ylabel("Fare ($)", fontsize=12)
plt.tight_layout()
plt.show()
```

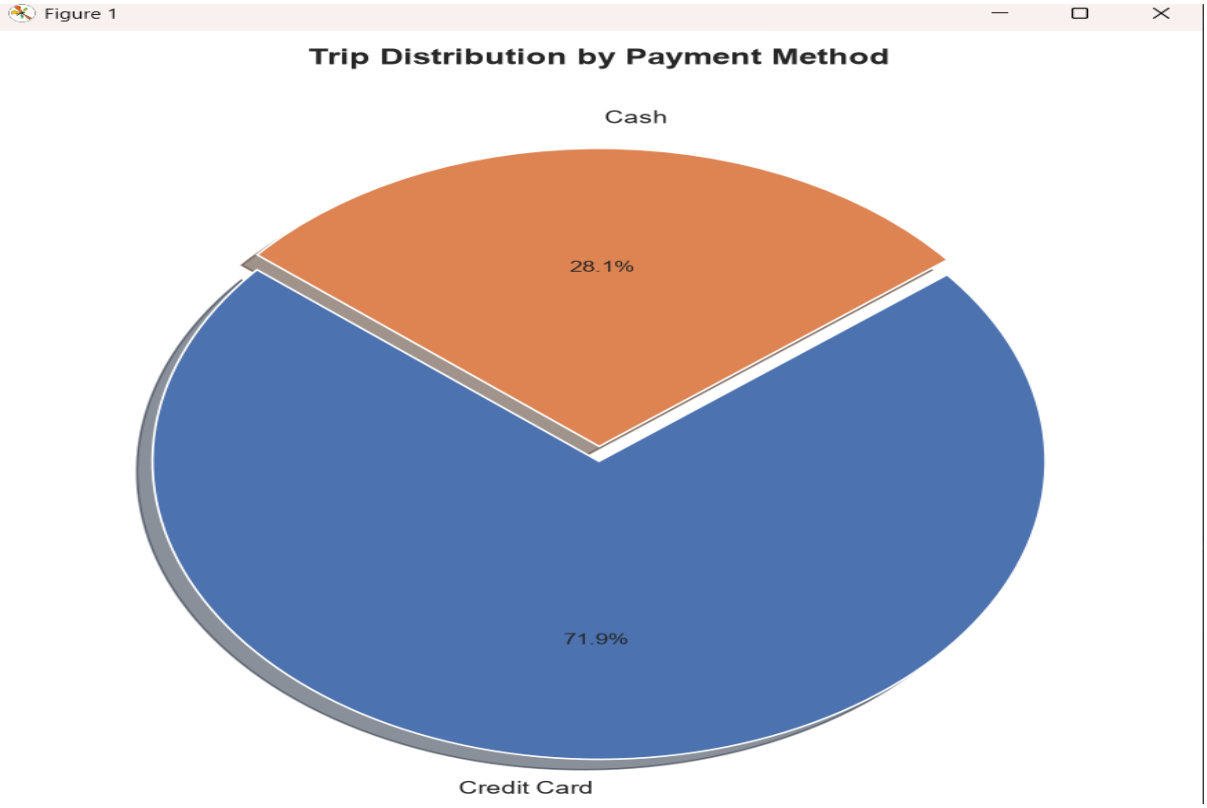
OUTPUT 3.1) LINE CHART: FARE OVER TIME



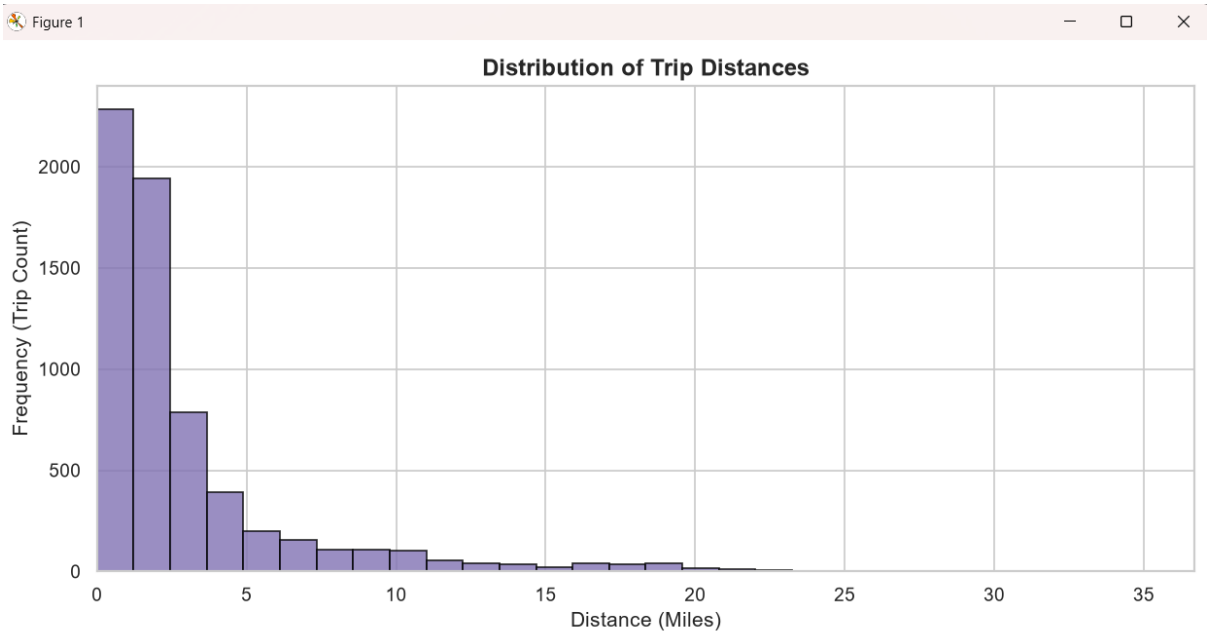
OUTPUT 3.2) BAR CHART: TOTAL FARE FOR EACH PICKUP BOROUGH



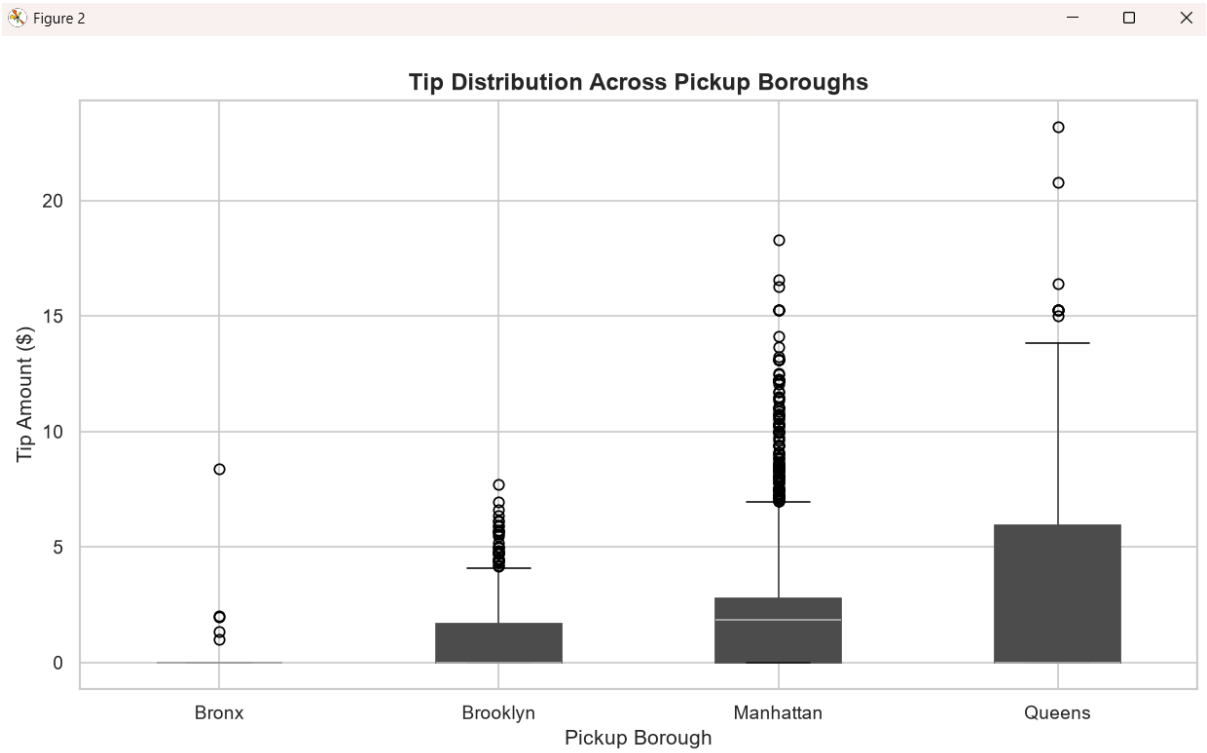
OUTPUT 3.3) PIE CHART: TRIPS BY PAYMENT METHOD



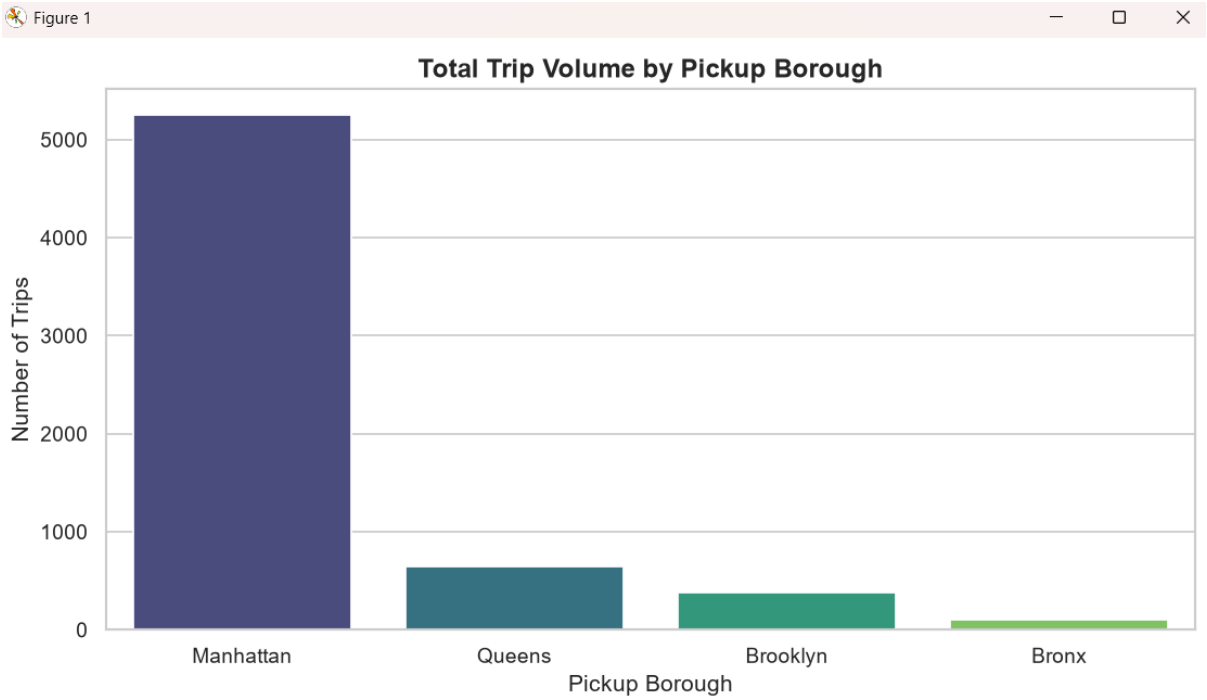
OUTPUT 3.4) HISTOGRAM: DISTRIBUTION OF TRIP DISTANCE



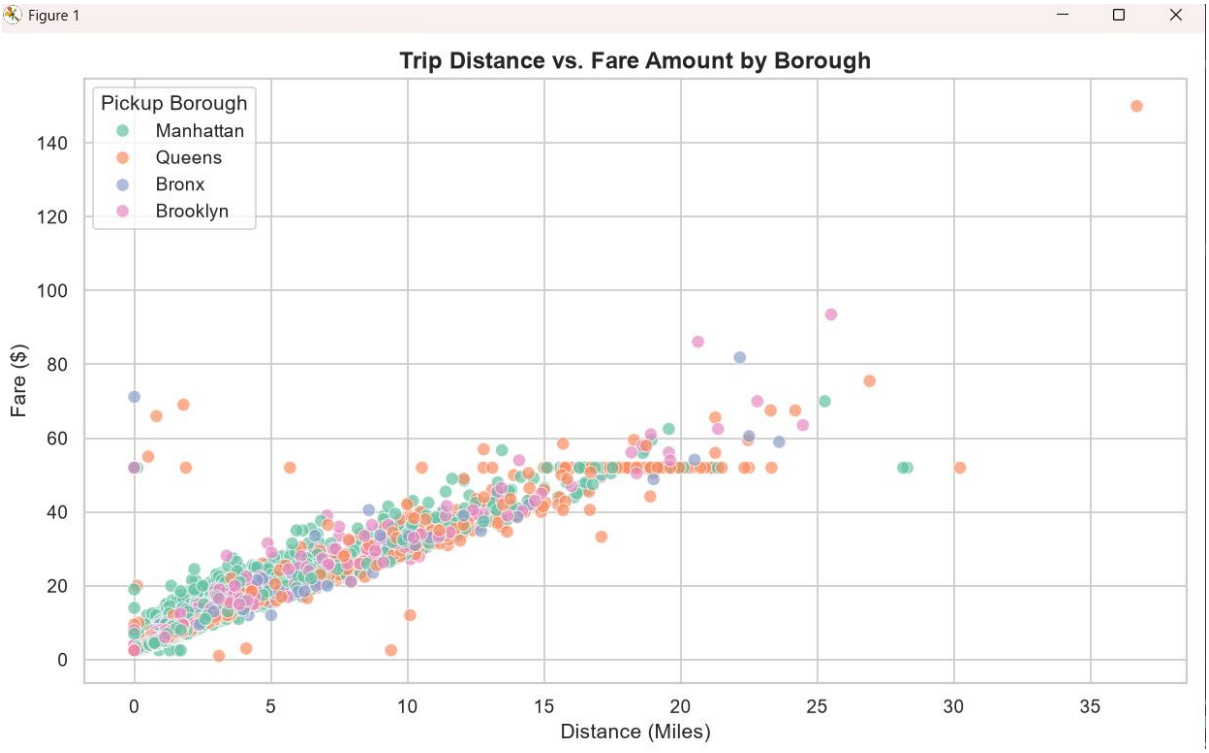
OUTPUT 3.5) BOX PLOT: TIP DISTRIBUTION BY PICKUP BOROUGH



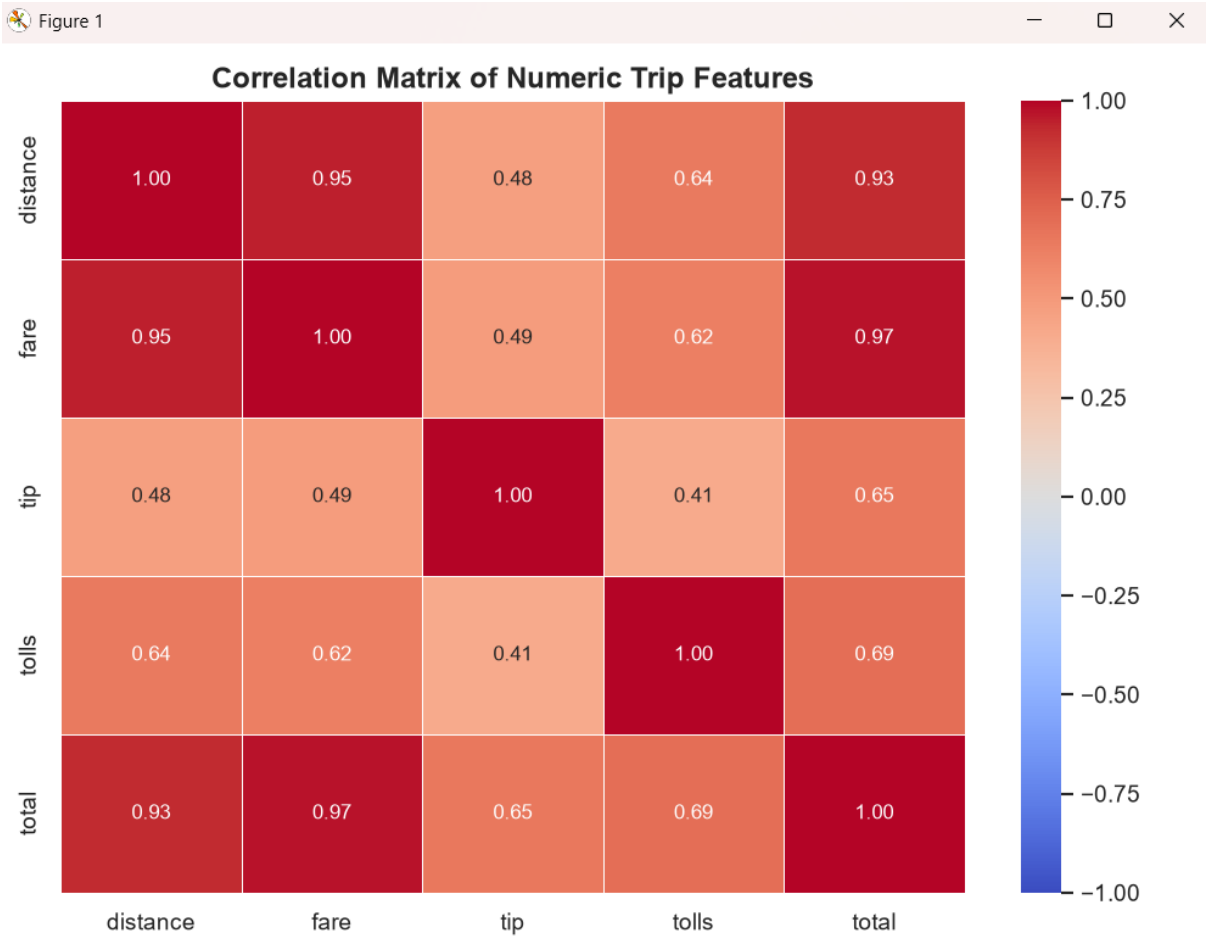
OUTPUT 4.1) COUNT PLOT: NUMBER OF TRIPS IN EACH PICKUP BOROUGH



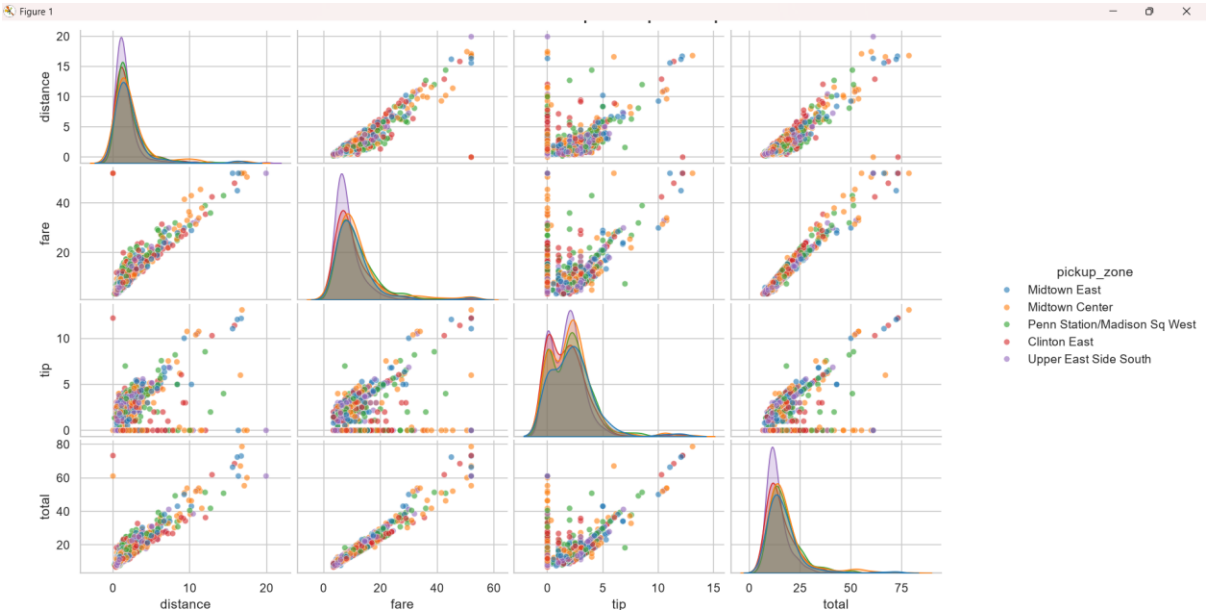
OUTPUT 4.2) SCATTER PLOT: DISTANCE VS FARE BY PICKUP BOROUGH



OUTPUT 4.3) HEATMAP: CORRELATION BETWEEN NUMERICAL VARIABLES



OUTPUT 4.4) PAIR PLOT: PAIRWISE RELATIONSHIPS BY PICKUP ZONE



OUTPUT 4.5) VIOLIN PLOT: FARE BY PAYMENT METHOD

